



Sense. Reason. Act.

Three personas, one LangGraph agent, 4.8 million rows of real MoveInSync operations data — detecting, explaining and acting [without being asked](#).

9

DETECTORS

4

COMPILED GRAPHS

5

LLM CALL SITES

3

PERSONAS, ONE GRAPH

A dashboard tells you **what**. Nobody has time to ask **why**.

MoveInSync moves 1M+ employees a day for 400+ organisations. The data streams 24/7 across timeliness, safety, cost, vendor performance and carbon. The bottleneck was never the data — it is the human who has to notice something, work out the cause, and decide what to do about it.

608,285

TRIPS

1,636,453

COMMUTE RECORDS

716,336

ATTENDANCE ROWS

51,699

SAFETY INCIDENTS

WHAT EXISTS TODAY**Charts, and a person**

Every number is relative only to itself. Was ₹114/km good? Is 87% SLA acceptable? The dashboard cannot say. Someone has to know.

WHAT BREAKS AT SCALE**Signal buried in volume**

608k trips a quarter produce far more anomalies than any transport manager can read. Alerting everything is the same as alerting nothing.

WHAT WE BUILT**An agent that closes the loop**

It notices, explains with evidence, decides who needs to know, drafts the action — and pauses for a human only where the stakes require it.

One agent, three jobs — and a chain of command between them

The requirement was to serve one persona. Serving all three only pays off if they are actually connected, so an unacknowledged alert escalates up the hierarchy on its own.

LEVEL 1 · LINE MANAGER

Team Commute Overview

“Is my team late because of the shuttle, or because of them?”

attendance_correlated

attendance_unrelated

LEVEL 2 · TRANSPORT MANAGER

Fleet Operations

“Which routes are breaching SLA right now, and who is unsafe tonight?”

delay_breach

incident

escort_violation

variability

LEVEL 3 · TRANSPORT HEAD

Strategic Overview

“How much are we overbilled, by whom, and are we hitting the carbon target?”

billing_discrepancy

emissions

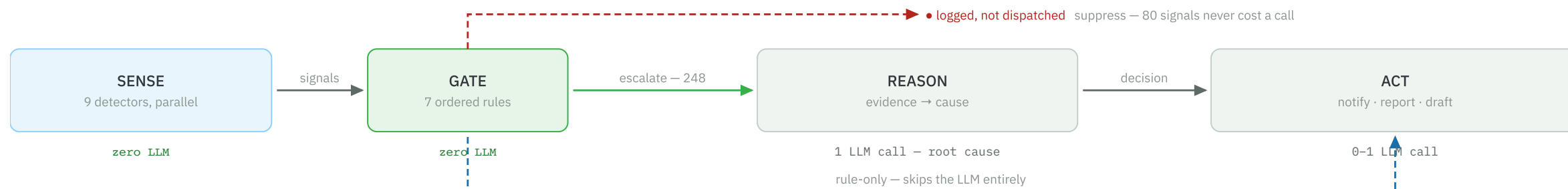
cost_divergence

```
# backend/app/graph/supervisor.py
ESCALATION_CHAIN = {
    "line_manager": "transport_manager",
    "transport_manager": "transport_head",
    "transport_head": None, # buck stops
}
```

Nothing gets dropped by being ignored. An open, unacknowledged notification past its severity timeout — critical 1h, high 4h — is re-issued to the persona above, titled [Escalated] ..., always immediate. info never escalates.

Sense → Gate → Reason → Act

The gate is the part that isn't in the brief. It sits **before** the LLM, not after, and decides whether a signal is worth thinking about at all.



The gate sits between detection and reasoning. Only signals that survive it reach a model — suppressed and rule-only paths cost nothing.

SENSE

9 detectors, parallel, pure SQL + thresholds.
Zero LLM.

GATE

7 ordered rules → suppress / rule-only / escalate. **Zero LLM.**

REASON

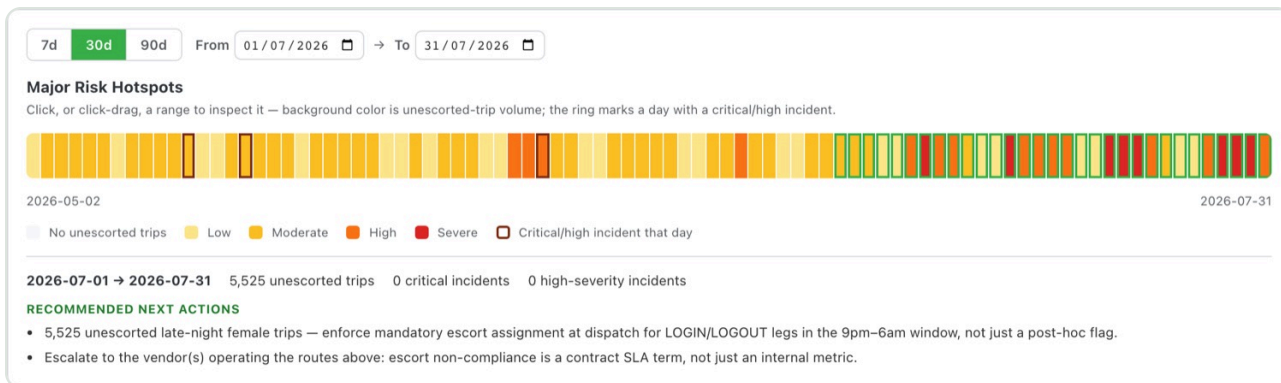
SQL agent, benchmark lookup, deterministic impact context, then root cause.

ACT

Notification, HTML report or drafted comms — with an interrupt() for sign-off.

The hotspot nobody was looking for

Late-night trips where a female employee travelled without the mandated escort. It is a contract SLA term, it is a safety exposure, and before this it was buried in 608k trip rows.



Live capture — 90 days of real trip data. Colour is violation density; the agent's own recommended actions sit under the strip.

THE RULE — DETECT_ESCORT_COMPLIANCE_SIGNAL

- Window **21:00 → 06:00**, LOGIN and LOGOUT legs
- `employee.gender = 'FEMALE'` and `trip.actual_escort = FALSE`
- Plus any open **panic event** — PANIC_MOBILE, WOMAN_TRAVELLING_ALONE
- critical** if the trip also ran ≥ 15 min late, otherwise **high**

5,525 unescorted late-night female trips surfaced in the demo window, and the org-wide compliance percentage is computed over the *unbounded* set — the 50-signal cap limits what gets dispatched, never what gets measured.

WHAT THE AGENT DOES WITH IT

This signal type is pinned to `safety_floor` — it can never be suppressed, batched or tuned away — and is routed to `communication`, which drafts a warning notice to the vendor's dispatch desk and **stops for human approval**.

The same lateness, two opposite conclusions

One detector splits a late arrival into a transport failure or an attendance problem — because telling those apart is the entire job, and getting it wrong penalises the wrong person.

DETECT_ATTENDANCE_CORRELATION

- ratio ≥ 0.60 → `attendance_correlated_with_transport`
defend the employee, escalate the delay pattern
- ratio ≤ 0.15 → `attendance_unrelated_late`
this one is worth a direct conversation
- Minimum 3 late samples; 25 signals per direction, worst first

The UI names the escalation target out loud: “escalate the underlying delay pattern to the Transport Manager rather than counting it against employees.”

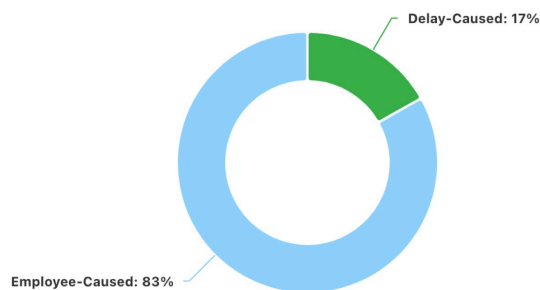
67%TRANSPORT-CAUSED
→ DEFEND**14%**

MIXED

0%EMPLOYEE-CAUSED
→ TALK

No-Show Cause Split

Delay-caused (shuttle late >15 min) vs. employee-caused



Team-wide split: only 17% of no-shows are shuttle-caused.

[Escalated] Employee**Watch**

- 160180 has been late 9 times...

Warning →

Employee 160180 has been late 9 times since 2026-07-24, but the data shows zero transport-caused incidents and no correlation with shuttle delays. -- escalated to transport_manager after 4h unacknowledged by line_manager.

Q How was this computed?

Per-employee verdict, in plain language, with the number that produced it.

Money on the table, named and quantified

Vendors bill by distance slab. The agent does not trust a rate card — it derives what each slab actually means from the org’s own billing history, then finds who is outside it.

DETECT_BILLING_DISCREPANCY_SIGNAL

- Slab bands from percentile_cont(0.5) per slab_name, min 20 samples, boundaries at adjacent-median midpoints
- Expected cost per slab needs ≥5 samples before it is used
- Vendor flagged at ≥3 trips and ≥₹500 exposure
- Severity by share of that vendor’s spend — ≥20% critical, ≥12% high

₹13.1 lakh

SLAB LEAKAGE · MAY–JUL · ALL VENDORS

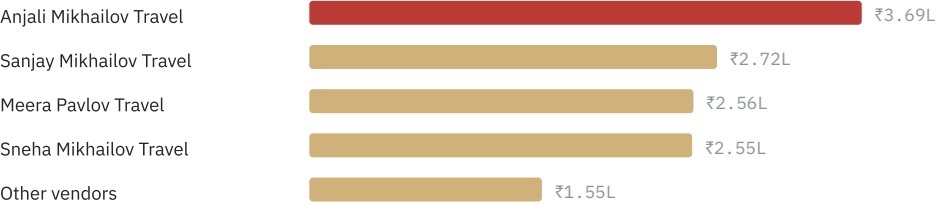
₹3.69 lakh

LARGEST SINGLE VENDOR

15-day audit cycle → under five minutes, and it runs every scheduler tick rather than once a cycle. The vendor scorecard turns that into a contract conversation.

Slab-tier leakage, May–Jul 2026

₹13.1 lakh total



Every rupee here is a trip billed at a higher distance slab than the distance it actually travelled — recovered by re-deriving each slab from the org’s own billing history.

Vendor Scorecard

SLA %, cost/km, incidents, weekly SLA trend

VENDOR	SLA %	COST/KM	INCIDENTS	VS PREVIOUS PERIOD	SLA TREND
Sneha Mikhailov Travel	87.1%	₹114.43	560	—	
Meera Pavlov Travel	85.6%	₹115.45	325	—	

Live capture. Sneha leads on the headline SLA and is marginally cheaper per km — and logs 72% more safety incidents over the same window. That tension is the renewal argument, and it only appears once incidents are attributed to the vendor who actually ran each trip.

Context is computed, not narrated

The brief asks that no metric arrives bare. We satisfy that in a node that never calls a model — so the context is always present, always consistent, and costs nothing.

```
# reason/state.py – attached to every decision
class ImpactContext(TypedDict):
    trend_direction: str # up | down | flat | unknown
    severity_band: str # low | medium | high | critical
    business_impact: str # one human sentence
    comparison_baseline: dict # what, and how, it was compared
```

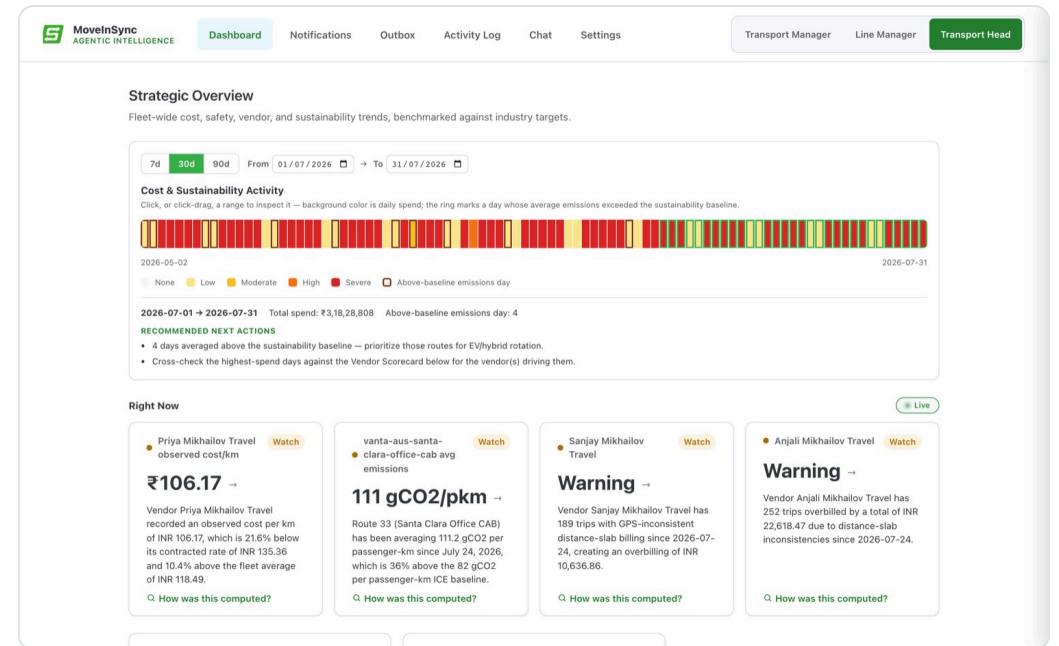
impact_context_builder is deliberately not an LLM call. Bands are arithmetic: flat inside $\pm 2\%$, then 15% / 40% / 75% of magnitude. A model can be unavailable, over budget or wrong — the context on a card can't be.

NOT THIS

Emissions: 111 gCO₂/pkm

THIS

“Route 33 has averaged **111.2 gCO₂ per passenger-km** since July 24 — **36% above** the 82 gCO₂ ICE baseline.”



Live capture. Every card on every persona carries its own context_note — the comparison, the direction and the business consequence, generated for the card, not typed by us.

109,280 data-quality problems, logged and contained

The real dataset is not clean. A tenth detector exists purely to find that — and, crucially, it reports to nobody.

PUBLIC.DATA_QUALITY_FLAGS — LIVE COUNTS

ISSUE TYPE	COUNT	WHERE IT COMES FROM
<code>null_required_field</code>	50,775	Drivers without a licence number; completed trips with no arrival time
<code>other</code>	31,385	Alert severities carrying the literal string "False", plus 16,348 nulls
<code>malformed_timestamp</code>	8,703	Arrival more than a day from its own trip date
<code>duplicate_row</code>	7,527	More than one completed trip per route per date
<code>orphaned_reference</code>	6,348	Bills and alerts pointing at trips that do not exist
<code>out_of_range_value</code>	4,542	Negative distances; passenger counts <0 or >100

flag_data_quality routes to no persona. It is the only detector whose persona tuple is empty — every finding is written and auditable, and not one of them ever becomes a human's inbox item. Data hygiene is an engineering backlog, not an operational alert.

FIXED ON THE WAY IN

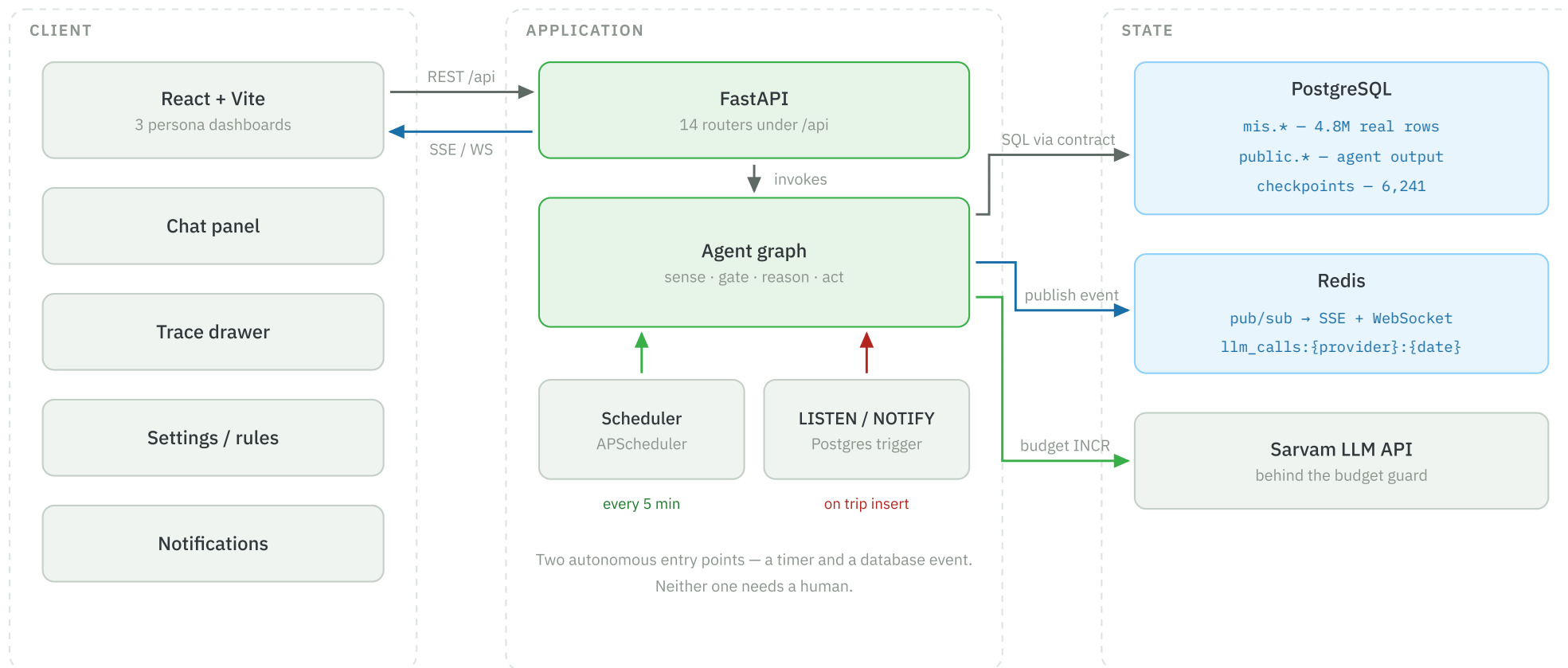
- Comma-formatted `trip_id` strings coerced to `int64`
- Placeholder `stwid = 0` employee rows filtered out
- Negative `planned_km` / `traveled_km` clipped to 0 *and flagged*
- Mixed epoch/ISO timestamps harmonised

FOUND LIVE, IN THE DATA

Some business units carry *every* late arrival on `mode= 'cab'` with zero `'shuttle'` rows. A bare `mode= 'shuttle'` filter silently zeroed the correlation for those orgs. The fix — and the reason for it — is a comment in the detector SQL the trace shows the judge.

System architecture

Seven containers. Two ways in — a 5-minute scheduler tick and a Postgres LISTEN/NOTIFY event — and one way back out to the browser.



The browser calls REST endpoints and receives server-sent-event / WebSocket pushes back. The agent reaches Postgres only through the data contract, and every LLM call increments a Redis counter before the request leaves the process. (A Cloudflare Tunnel container, omitted here, optionally exposes FastAPI publicly.)

No node in this codebase knows a table name

Every entity and column a detector, chart or metric touches is resolved through one mapping file. That is the whole answer to “would this survive contact with your production schema?”

```
# backend/config/data_contract.yaml - 397 lines, version 2
entities:
  trip:
    table: mis.trip
    columns:
      scheduled_time: scheduled_arrival
      actual_time: actual_arrival
      actual_escort: actual_escort
      traveled_km: traveled_km
# and in every node, without exception:
contract = get_contract().entity("trip")
table, c = contract.table, contract.column
SELECT {c('actual_time')} FROM {table} ...
```

Repointing the whole system at MoveInSync’s real schema is one file and zero code changes — set DATA_CONTRACT_PATH at a different mapping and every detector, chart and report follows. 17 of 17 entities validate on boot.

WHAT THAT BUYS, CONCRETELY

- The SQL agent’s table allow-list is *derived* from the contract — checkpoint, store and pipeline tables are invisible to the model by construction
- Column renames in the warehouse never reach application code
- The same agent runs on a synthetic schema and the real `mis.*` one

17/17

ENTITIES VALID

0

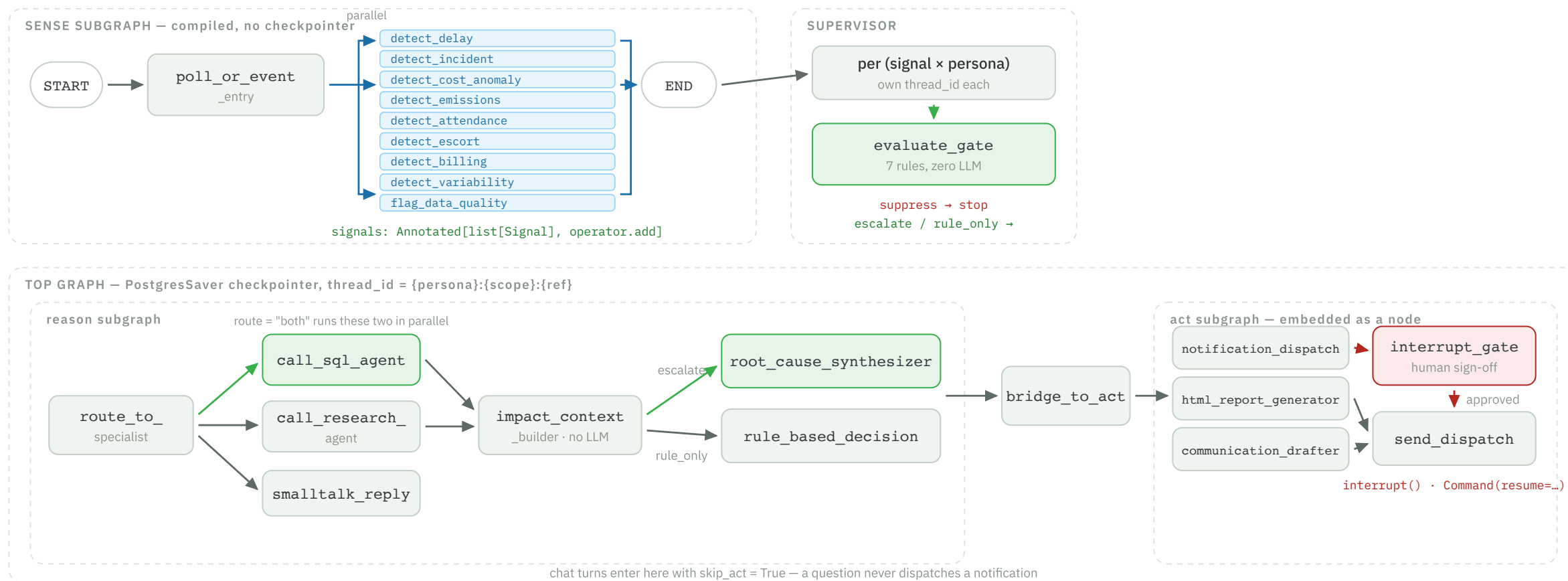
HARDCODED TABLES

1

FILE TO REPOINT

LangGraph agentic architecture

Four compiled StateGraphs. Act is embedded in the top graph as a real subgraph node, so `interrupt()` and `Command(resume=...)` persist on the parent thread.



Four compiled StateGraphs. Nine detectors write concurrently into one reducer; the gate decides whether reasoning runs at all; act is a real subgraph node, so its interrupt survives on the parent thread.

Graph state — the contract between nodes

Nothing is passed as a loose dict. Each subgraph declares what it consumes and what it is allowed to add, and the reducer on `signals` is what makes nine parallel detectors safe.

SENSESTATE — FAN-OUT PRODUCER

```
org_id: str
since: datetime | None
event: dict | None
signals: Annotated[list[Signal], operator.add]
dq_issues_found: int

# the reducer above is the whole reason nine
# detector nodes can write concurrently
Signal: signal_type · entity_type · entity_id
      severity · summary · raw_metric · source
```

REASONSTATE — ROUTING + EVIDENCE

```
gate_mode · gate_reason · gate_confidence
route: "sql" | "research" | "both" | "context_only"
sql_result:      SQLAlchemyResult | None
research_result: ResearchComparison | None
impact_context:  ImpactContext | None
decision:        ReasonDecision | None

ReasonDecision: summary · root_cause · recommendation
                confidence · needs_human_signoff · target_persona
                supporting_evidence
```

SQLAGENTSTATE — SELF-CORRECTING LOOP

```
messages: Annotated[list[AnyMessage], add_messages]
question · org_id · table_names · schema_context
generated_sql: str
sql_retry_count: int          # hard cap 3
query_error_log: list[str]    # fed back to the model
result_rows · final_answer · success · done
out_of_scope · out_of_scope_reason
```

ACTSTATE — ROUTING + OUTCOME

```
action_type: "notification" | "report" | "communication"
severity: "info" | "warning" | "critical"
scope · thread_id · audience · notification_cadence
# ---- written back out ----
notification_id · notification_status · report_id
communication_draft · approval · dispatch_status
```

Nine detectors, and every number in them is configurable

DETECTOR	FIRES WHEN	ESCALATES TO CRITICAL	GOES TO
delay_breach	≥25% of route trips breach 15 min , or avg delay ≥20 min (≥3 trips)	avg ≥35 min or ≥80% breached	Transport Mgr
incident	Unresolved and severity ≥ medium	severity = critical	Transport Mgr
escort_compliance_violation	Unescorted female trip, 21:00–06:00 , or open panic event	trip also ≥15 min late	Transport Mgr
performance_variability	Coefficient of variation ≥ 20% over ≥15 samples, 10-min floor	CV ≥ 2× threshold → high	Transport Mgr
cost_divergence	≥ 20% off contract rate or fleet average (≥3 trips)	≥50% divergence	Transport Mgr + Head
billing_discrepancy	≥3 flagged trips and ≥ ₹500 , slab bands from ≥20 samples	≥20% of vendor spend	Transport Head
emissions_over_target	≥ 1.05× the 82 gCO ₂ /passenger-km baseline	≥1.5× baseline	Transport Head
attendance_correlated	Transport-correlation ratio ≥ 0.60 (≥3 lates)	ratio ≥0.85 → high	Line Manager
attendance_unrelated_late	Correlation ratio ≤ 0.15	—	Line Manager
flag_data_quality	Nulls, duplicates, malformed timestamps, out-of-range values	—	nobody, by design

None of these are constants in code. Each is a default that an alert_rules row overrides per organisation and per signal type — JSON params, gate mode and notification cadence — cached for 30 seconds and editable from the Settings page, live within one scheduler tick. A 7-day lookback anchors on the last date with real volume, not wall-clock.

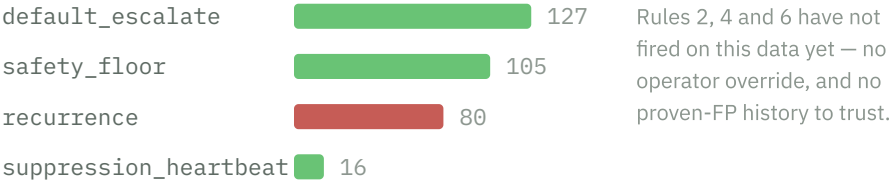
Seven rules decide whether an LLM is worth it

First match wins. Every decision — including every suppression — is written to `gate_decisions` and shows up in the trace.

#	RULE	ACTION	CONF.
1	<code>safety_floor</code>	escalate incidents, escort violations, anything critical — not overridable	1.00
2	<code>operator_override</code>	forced an operator’s explicit Settings choice	1.00
3	<code>suppression_heartbeat</code>	escalate 5 consecutive suppressions — nothing hides forever	0.90
4	<code>insufficient_sample</code>	suppress below the detector’s minimum sample size	0.70
5	<code>recurrence</code>	suppress already dispatched 3× in 24h	0.60
6	<code>high_margin_breach</code>	rule-only ≥2× over threshold and proven FP rate ≤20%	0.50–0.95
7	<code>default_escalate</code>	escalate when nothing else matched	0.50

328 gate decisions logged

24.4% never reached a model



Live counts from `public.gate_decisions` on the running instance.

The safeguards are the interesting part. A scope can never be suppressed indefinitely — rule 3 has already fired **16 times** on this data, forcing suppressed scopes back into view. And `rule_only` demands a *proven* low false-positive rate: a signal type with no dispatch history yet always escalates. Unknown is never treated as safe.

Five LLM call sites in the entire system

Everything that can be arithmetic is arithmetic. The model is spent on the two things it is genuinely better at: writing SQL, and explaining a cause in a sentence a human will act on.

CALLS THE MODEL · 5 SITES

<code>generate_query</code>	Text-to-SQL, grounded on live DDL + 3 sample rows · 8192 tok
<code>synthesize_answer</code>	Answers strictly from returned rows · 2048 tok
<code>root_cause_synthesizer</code>	Signal + context + SQL evidence → cause & recommendation · 1024 tok
<code>communication_drafter</code>	Audience-tuned notice, under 120 words · 12288 tok
<code>report_narrative</code>	Leadership summary over a period · 12288 tok

Structured output is prompt-and-parse (fenced JSON → regex → `json.loads`), never tool-calling — a deliberate choice for a non-frontier model.

NEVER CALLS THE MODEL

- all 9 detectors
- poll_or_event_entry
- evaluate_gate
- route_to_specialist
- impact_context_builder
- rule_based_decision
- smalltalk_reply
- bridge_to_act
- check_query (the guard)
- every act router

Routing is a keyword check, not a model call. Asking a model which specialist to use would double the cost of every turn to answer a question a `frozenset` already answers.

MODEL

sarvam-105b-conversations over an OpenAI-compatible endpoint. Provider and model are environment variables — `LLM_PROVIDER` / `LLM_MODEL` — not an import.

The budget guard, caught doing its job

A circuit breaker only counts if it has actually tripped. Ours did — in this instance, today — and the system degraded instead of falling over.

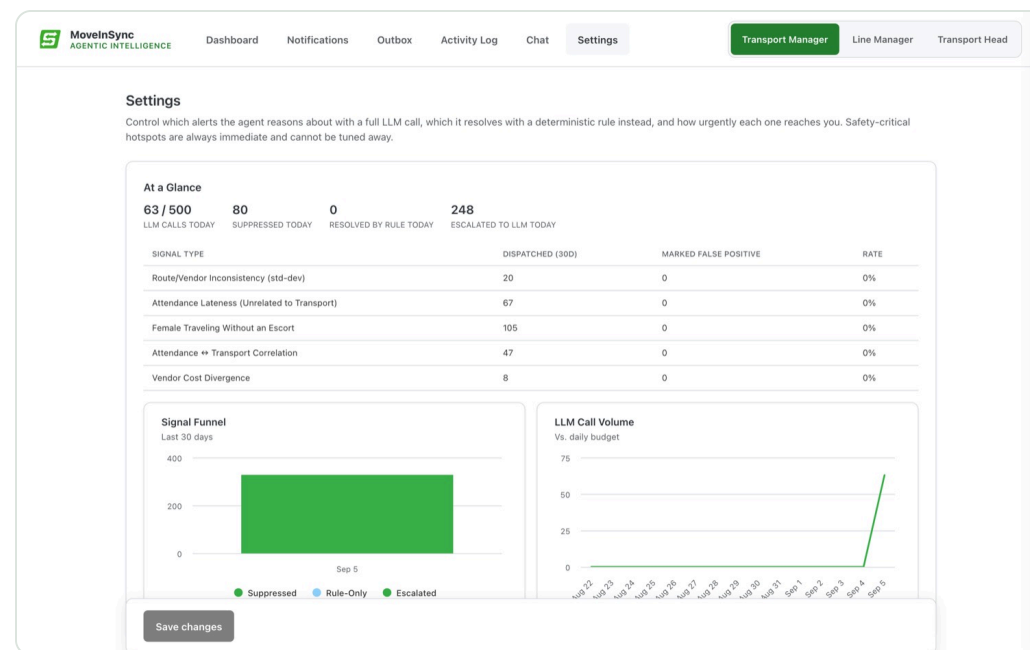
```
# llm/provider.py — a subclass, not a wrapper
class _BudgetGuardedChatOpenAI(ChatOpenAI):
    # overrides _generate / _agenerate / _stream / _astream
    # so bind_tools() and with_structured_output()
    # physically cannot route around the check

    INCR llm_calls:{provider}:{date}    # TTL 26h
    if count > daily_limit:
        raise LLMBudgetExhaustedError    # before the HTTP call
```

The counter increments before the request leaves the process, so a crash mid-call cannot under-count. Over budget, the API returns **503** — never a fabricated 200.

WHAT ACTUALLY HAPPENED HERE

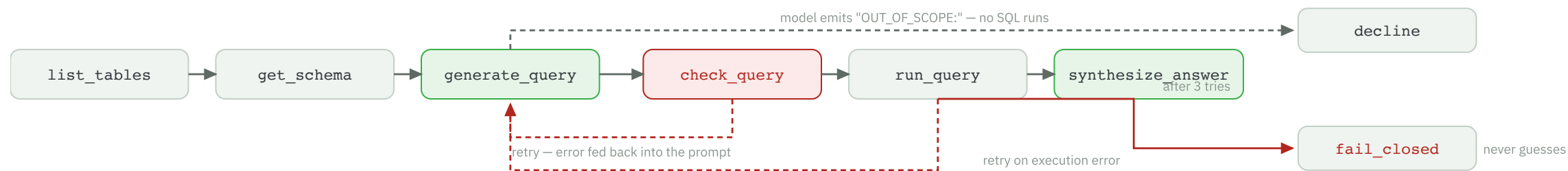
The scheduler hit the 500-call ceiling. **90 escort-compliance signals** were refused at `root_cause_synthesizer` with `LLMBudgetExhaustedError` — and each one was logged as “will retry automatically next cycle,” not lost, not half-written, not crashed.



Live capture. Budget, suppression and false-positive rate per signal type are operator-facing — and the copy is explicit that safety hotspots “cannot be tuned away”.

A model writes the query. It is not trusted to.

Text-to-SQL over a production warehouse is the highest-risk thing in this system, so the guarantee is structural rather than instructional.



A hand-wired node loop, not a tool-calling ReAct agent — which is what makes the retry cap and the error log inspectable in the trace.

THE HARD BOUNDARY — SECURITY.PY

- Parsed with `sqlglot` into an AST, never regex
- Rejects more than one statement — no ; stacking
- Rejects anything that is not `exp.Select`
- Walks the whole tree rejecting `Insert` · `Update` · `Delete` · `Drop` · `Alter` · `Create` · `Truncate` · `Grant` · `Command`

BLAST RADIUS

- **LIMIT 200** injected if absent, lowered if higher, re-capped after execution
- `statement_timeout = 8000ms` set on the connection itself
- Table allow-list derived from the data contract — checkpoint and internal tables are invisible
- `org_id` filter required by prompt and schema context

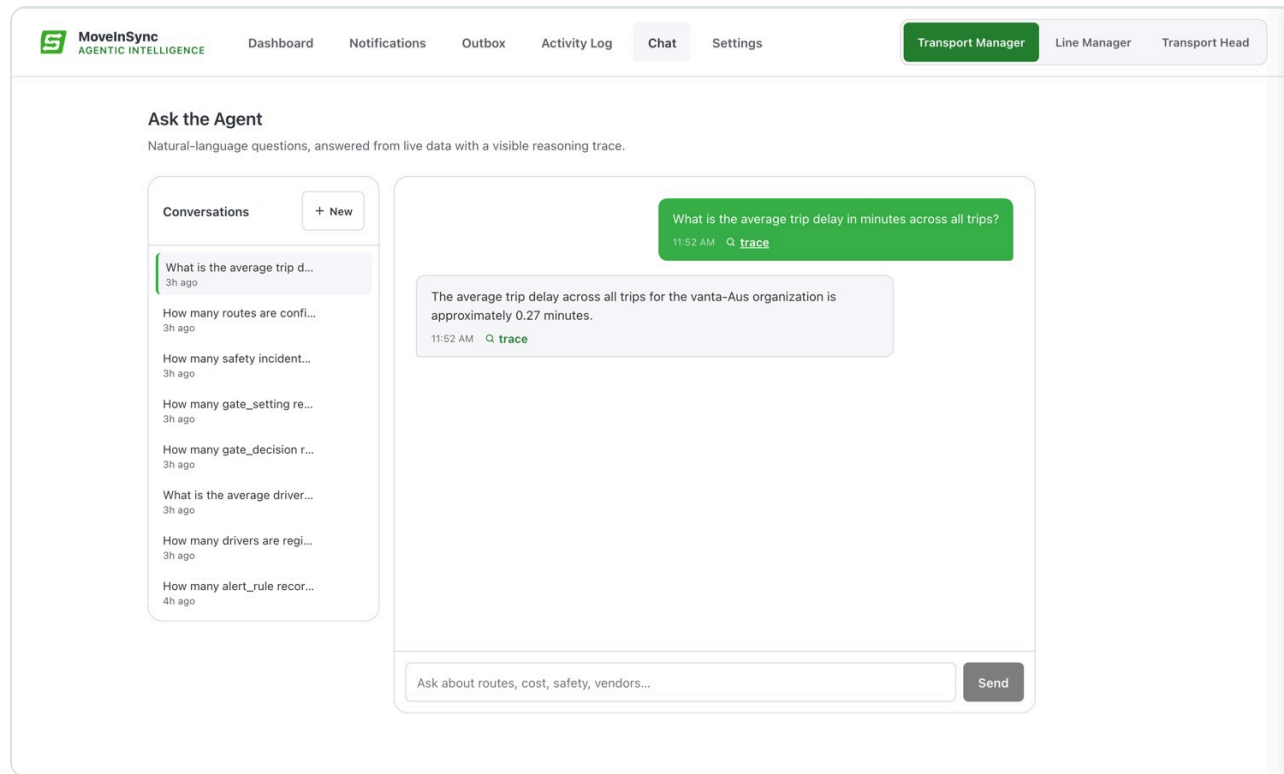
FAILING HONESTLY

- **3 retries**, each fed the previous SQL and every error, then `fail_closed` — it never guesses
- A truncated generation counts as a retry rather than being parsed as SQL
- Out-of-scope questions hit `decline` — no SQL is ever executed

Prompt injection is treated as expected input, not an edge case: the question is passed as data, and **the guard — not the instruction — is what makes a write impossible**. A jailbroken model still cannot reach a mutating statement.

Ask it anything the warehouse can answer

The same graph, entered without a signal. POST /api/chat runs reason with skip_act=True, so a question never dispatches a notification.



Live capture — answered from 1.6M commute rows, with a trace link on every message.

ONE IDENTIFIER, THREE JOBS

`chat_threads.id` is the LangGraph checkpoint `thread_id` is the episodic-memory `user_id`. Memory is namespaced ("memories", `persona`, `thread_id`, "episodic"), so forgetting a conversation is exact — one thread, nothing else.

HONEST FAILURE

- Budget exhausted or provider unconfigured → **503**
- Anything else → **502**
- Never a 200 carrying an invented answer

Small talk is short-circuited by a keyword check before any node runs — greeting the agent costs a database round trip of exactly zero and an LLM call of exactly zero.

Every claim on screen can be opened

“How was this computed?” is on every card and every chat message. It replays the agent’s own checkpoint history — **no separate instrumentation was added to produce it.**

HOW WAS THIS COMPUTED? Chat Answer Trace

- Signal Detected** 11:52 AM
Chat question received: "What is the average trip delay in minutes across all trips?"
- SQL Generated**
Text-to-SQL agent grounded on live schema DDL + sample rows,

```
SELECT AVG(delay_minutes) AS average_trip_delay_minut
```
- SQL Executed** 11:52 AM
Returned 1 row(s).
- Context Attached** 11:52 AM
What is the average trip delay in minutes across all trips? -- low severity.
- Decision** 11:52 AM
The average trip delay across all trips for the vanta-Aus organization is approximately 0.27 minutes. Root cause: The low average delay of 0.27 minutes for vanta-Aus suggests that, at a high level, trips are running close to schedule; no specific operational driver or incident is indicated by the data provided. Recommendation: Verify whether this baseline aligns with historical trends for vanta-Aus before deciding on any operational adjustment. Confidence: 95%

RECOMMENDED ACTION
Verify whether this baseline aligns with historical trends for vanta-Aus before deciding on any operational adjustment.

THE SEVEN STEP KINDS

- **Signal Detected** — carries the *detector's own* SQL, stashed in `raw_metric`
- **Gate Evaluated** — action, reason, matched rule, confidence
- **SQL Generated** — the query, and the retry count
- **SQL Executed** — rows returned, self-corrections, last error
- **Impact Context Attached** — the business-impact sentence
- **Decision** — cause, confidence, and a separate recommended action
- **Escalated** — recurses into the original thread's history

`build_trace()` is a formatter. It reads `get_state_history()` off the checkpointer and reaches outside it exactly once — for the gate row, which is written before the graph is invoked. Transparency is a property of the architecture, not a feature bolted onto it.

Live capture — the generated SQL, the rows it returned, and the confidence behind the recommendation.

It acts on its own — except where it shouldn't

Autonomy is the point, but a warning notice to a vendor's dispatch desk is not something a model should send unsupervised. So the graph stops.

```
# act/nodes.py — a real LangGraph pause, not a flag
approval = interrupt({
    "type": "approval_request",
    "draft": communication_draft, ...
})
# the graph halts here; state is checkpointed to Postgres.
# resumed later, from anywhere, on the same thread_id:
graph.invoke(Command(resume={"approved": True,
                          "approver": user}), config)
```

Re-clicking Approve cannot double-send. Idempotency comes from the thread's checkpoint, not from a status column we remember to check — the node replays from the top and the `interrupt()` resolves once.

WHO GETS STOPPED

- Incidents and escort violations — routed to communication, always sign-off
- Everything else dispatches autonomously as a notification
- The scheduler generates daily, weekly and monthly reports on its own cadence

The screenshot shows the 'Activity Log' section of the MoveInSync interface. It features a navigation bar with tabs for Dashboard, Notifications, Outbox, Activity Log (selected), Chat, and Settings. On the right, there are buttons for Transport Manager, Line Manager, and Transport Head. The main content area is titled 'Activity Log' and includes a description: 'A read-only audit trail of every autonomous pipeline run, across ALL personas — not a to-do list (that's Notifications). Each row fired on its own, on a schedule tick or a live data event, never a click, and shows what the agent found and decided.' Below this, it says 'Showing 25 of 307 runs'. The table lists several runs, all initiated by the 'Transport Manager' at 'Sep 5, 03:03 PM'. The first three runs are marked as 'Processed a signal for this scope.' The subsequent five runs are marked as 'Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.' and each has a 'Retrying' status indicator.

Icon	Transport Manager	Scheduled run		Sep 5, 03:03 PM
🕒	Transport Manager	Scheduled run	Processed a signal for this scope.	Sep 5, 03:03 PM
🕒	Transport Manager	Scheduled run	Processed a signal for this scope.	Sep 5, 03:03 PM
🕒	Transport Manager	Scheduled run	Processed a signal for this scope.	Sep 5, 03:03 PM
🕒	Transport Manager	Scheduled run	Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.	Sep 5, 03:03 PM Retrying
🕒	Transport Manager	Scheduled run	Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.	Sep 5, 03:03 PM Retrying
🕒	Transport Manager	Scheduled run	Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.	Sep 5, 03:03 PM Retrying
🕒	Transport Manager	Scheduled run	Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.	Sep 5, 03:03 PM Retrying
🕒	Transport Manager	Scheduled run	Couldn't finish reasoning about an escort-compliance issue this cycle -- will retry automatically next cycle.	Sep 5, 03:03 PM Retrying

Live capture — the agent's own activity log. Every entry here was produced by a scheduler tick or a database event, with nobody at the keyboard.

Watch it react to something that just happened

Real trips from the provided dataset, re-inserted with today's timestamp. No mock events, no scripted animation.

```
python backend/db/real_data/replay.py --scenario delay_spike --count 5 --interval-seconds 3
```

1 · INSERT

Postgres LISTEN/NOTIFY fires on the new row.

2 · SENSE → GATE

Detector matches; the gate rules on it and logs why.

3 · REASON → ACT

SQL evidence, root cause, dispatch decision.

4 · PUSH

Redis → SSE → the card appears. Nobody clicked.

REQUIREMENTS

- Runs on the real dataset
- Sense → reason → act
- Serves a persona (all three)
- Every metric carries context
- Handles messy data
- Triggers on its own
- Combines 6 solution forms
- Leadership output, untouched

ALL SIX ACCEPTED SOLUTION FORMS, IN ONE SYSTEM

Conversational agent · Proactive alerting & triggers · Automated reporting & narratives · Insight & anomaly detection · Decision-support dashboard · Automated communications